

AI Collectives in gem5 Garnet

Multicast, Bypass Links, and Real-H100 Trace Replay

Lab 4 Project Team

August 7, 2026

Abstract

This report evaluates three related interconnection mechanisms implemented in gem5 Garnet: router-level tree multicast, express-link bypass routing, and trace-driven broadcast/all-reduce traffic. The multicast regression passes 208 paired correctness cases and its 162-case performance matrix reduces internal-link flit traffic by 39.2% on average with a $3.59\times$ median completion-latency speedup, while retaining ten negative throughput cases. The bypass study shows that optimistic shortcut latency is an upper bound: distance-scaled links can regress, and a tree-sharing-aware policy must reject shortcuts that destroy multicast reuse. Finally, an executed eight-H100 NCCL microbenchmark drives 30 broadcast requests (6,720 flits) and 6,720 scalar all-reduce lanes (53,760 rank contributions) to exact completion on paired Mesh and Bypass topologies. These are mechanism and scaled-trace results, not measurements of a complete model, NVLink/NVSwitch, or vector reduction hardware.

1 Scope and Research Question

The multicast portion of Topic 3 asks whether one source can deliver a packet to multiple destinations with one logical injection instead of transmitting one complete copy per destination. We answer the following question:

For the same source, destination set, payload, request schedule, and network configuration, how does router-level tree replication compare with source-side replicated unicast in traffic, completion latency, and throughput?

This distinction is important. A broadcast that always visits every router does not implement arbitrary multicast, and an all-reduce broadcast is not a fair multicast baseline. Our implementation carries a 64-bit destination bitmap, prunes unused branches, and generates a local copy only when the local router belongs to the destination set. The current bitmap limits one multicast domain to at most 64 routers.

The bypass portion asks whether added long-range links can reduce ordinary Mesh traversal without ignoring wire length, radix, buffering, or multicast tree sharing. Topic 4 asks whether a collective trace captured on real GPUs can deterministically drive the same Garnet mechanisms with auditable scaling and exact completion checks. We report these questions separately because a correct mechanism, a synthetic performance result, and a trace replay provide different levels of evidence.

2 Architecture and Baseline

2.1 Replicated-unicast baseline

For a logical destination set of size N , the source Network Interface (NI) creates one normal deterministic-XY packet for every remote destination. A local destination completes locally and creates no internal-link traversal. Each physical packet uses the standard Garnet input buffering, route computation, VC allocation, switch allocation, and credit flow control. All copies retain the same logical multicast identifier and round identifier.

2.2 Tree multicast

The source injects one physical packet. Its head flit carries the destination bitmap and establishes persistent output-branch state at each router. Body and tail flits use the same branch set. A router sends a flit only when

all selected branches have the required VC or credit, preventing a partially replicated flit from being dropped or duplicated. The tail releases branch VCs and per-packet state. This atomic-fanout behavior is conservative: it preserves correctness under backpressure but can make a fast branch wait for a congested branch.

2.3 Common completion contract

A logical request completes only when every expected destination has received exactly one complete packet with the correct identifier and payload. Missing, duplicate, unexpected, or wrong-round delivery is fatal. Reaching the simulation tick limit is not accepted as successful completion.

3 Experimental Method

Table 1 summarizes the performance matrix generated by `run_multicast_performance.py`. Every row is run once with replicated unicast and once with tree multicast using identical parameters. Warmup and cooldown requests are excluded from measured latency and throughput.

Table 1: Current paired performance matrix.

Parameter	Values
Topology	4×4 Mesh, deterministic XY tree
Source router	5
Destination count	4, 8, 16
Packet size	1, 4, 16 flits
Offered request probability	0.1, 0.5, 1.0
Background unicast rate	0.0, 0.1
Random seeds	1, 7, 17
Phases	20 warmup, 100 measured, 20 cooldown requests
Maximum outstanding requests	8
Paired cases / raw runs	162 / 324

The derived metrics are

$$\begin{aligned}
S_L &= L_{\text{naive}}/L_{\text{tree}}, \\
R_T &= 1 - F_{\text{tree}}/F_{\text{naive}}, \\
I_Q &= Q_{\text{tree}}/Q_{\text{naive}} - 1,
\end{aligned}$$

where L is logical-request completion latency, F is measured internal-link flit traversal, and Q is completed logical multicasts per cycle. Absolute values are retained in the released CSV and JSON files.

4 Correctness Evidence

The current correctness command reruns both modes for 26 source/destination configurations and four packet sizes, producing 208 paired cases. It includes all 15 nonempty destination masks in a 2×2 network, source-included and source-excluded cases, local-only delivery, row/column groups, sparse groups, and random groups on 3×3 and 4×4 meshes. Packet sizes are 1, 4, 16, and 64 flits. All 208 cases passed in the audited run on August 7, 2026.

The runner checks the final destination bitmap, completion-driven exit, logical request count, destination delivery count, source flits, router-generated flits, physical packet count, and zero reduction merges. It therefore checks exact small-topology traffic invariants in addition to successful termination.

5 Performance Results

5.1 Physical traffic

Figure 1 shows that traffic reduction grows with multicast fanout. The average reduction is 29.1% for four destinations, 35.3% for eight destinations, and 53.1% for all 16 destinations. The overall mean is 39.2%, with

a range from 11.1% to 53.1%. Variation at group size four is caused by the geometry of the randomly selected destination set: destinations with more overlapping XY paths provide more replication reuse.

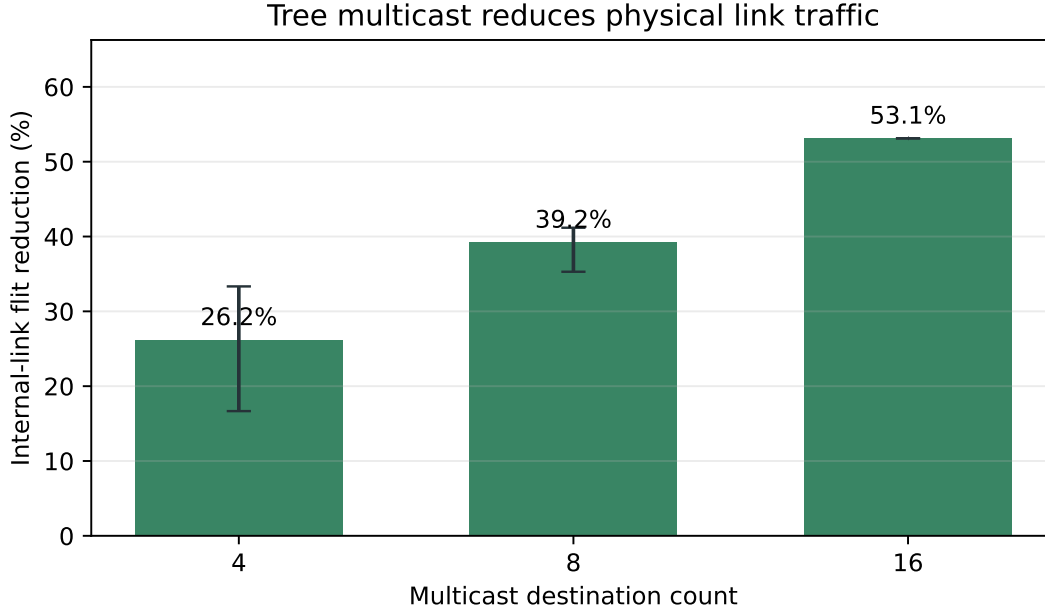


Figure 1: Mean internal-link flit reduction. Error bars show the observed minimum and maximum across paired cases, not a confidence interval.

5.2 Completion latency

Figure 2 reports latency speedup grouped by fanout and packet size. Every measured paired case has $S_L > 1$, with an overall mean of $4.79\times$, median of $3.59\times$, and range of $1.20\times$ to $22.51\times$. The increasing benefit with destination count is expected: replicated unicast serializes or contends among more source-generated copies, while the tree shares common path prefixes.

The very large maximum for four-flit packets should not be generalized beyond this matrix. It combines particular destination placement, bounded outstanding requests, and source-side replication pressure. The final study must retain per-case results instead of reporting only the mean.

5.3 Throughput and contention

Figure 3 shows the mean throughput change by fanout and packet size. Full-group multicast benefits strongly because it removes many source-generated copies. Small-group, long-packet traffic benefits much less: the four-destination, 16-flit cell improves by only 1.5% on average.

Figure 4 gives absolute throughput for full-group multicast with no background traffic. The 0.5 and 1.0 offered-probability points frequently coincide. With at most eight outstanding requests, these points exercise a bounded-outstanding regime rather than providing a finely resolved saturation curve. Additional offered-load points and accepted-load reporting are needed for the final saturation analysis.

The overall mean throughput change is $+190.9\%$ and the median is $+121.7\%$, but the distribution is skewed by high-fanout cases. As shown in Figure 5, 10 of 162 cases (6.2%) regress. All regressions occur for four-destination groups; the worst case is a 16-flit packet with seed 17, no background traffic, and offered probability 0.5 or 1.0. It has 33.3% less link traffic and $1.55\times$ lower completion latency, yet its steady-state throughput is 18.2% lower. This is consistent with the atomic fanout policy: reduced traffic does not guarantee that every branch can make progress independently.

Tree-mode credit stalls are nonzero in 45 of 162 raw tree runs, with a maximum of 411 recorded stalls. This confirms that the performance matrix exercises backpressure, although it does not replace an explicit low-buffer/one-VC stress matrix.

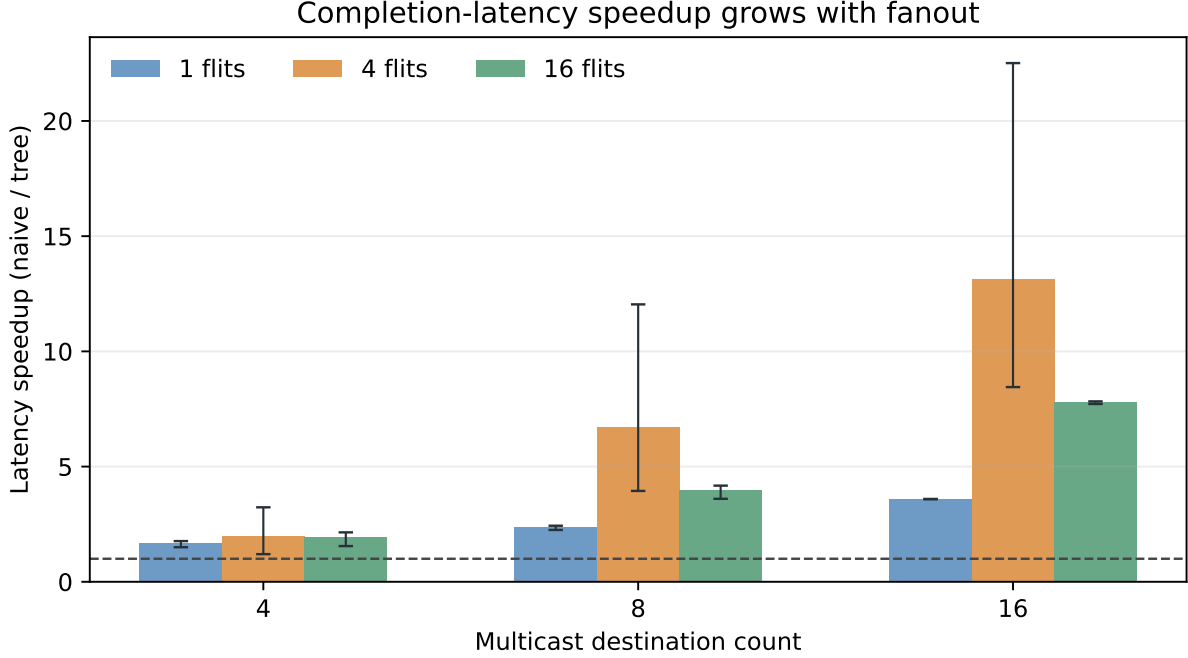


Figure 2: Completion-latency speedup. Bars are arithmetic means and error bars span the observed minimum and maximum. The dashed line is break-even.

6 Bypass Links and Multicast Interaction

The bypass extension augments the ordinary Mesh with diagonal or stride express links. Each placement is evaluated with an optimistic one-cycle model and a distance-scaled model whose serialization/latency reflects the link’s Manhattan span. The G9 matrix contains 7,680 successful gem5 runs, 6,144 paired seed cases, two Mesh sizes, four traffic patterns, four packet sizes, 16 offered-load points, and three seeds.

Table 2: Mean bypass results over the full synthetic matrix.

Design	Latency speedup	Throughput change	Traversal change
Diagonal, distance-scaled	1.629×	+6.41%	−2.82%
Diagonal, optimistic	1.723×	+10.61%	−7.03%
Stride, distance-scaled	0.981×	−1.94%	+15.50%
Stride, optimistic	1.103×	+3.68%	+10.10%

The aggregate table alone is not a hardware claim. At offered load at most 0.1, diagonal and stride distance-scaled mean latency speedups are only 0.962× and 0.936×, respectively. Diagonal placement also adds 9 undirected links, wire-length proxy 18, and maximum Router radix 8 on the 4×4 Mesh; the corresponding 8×8 values are 49, 98, and 8. Thus fewer Router traversals do not guarantee lower latency after long-wire and port costs are represented.

G10 explicitly tests multicast interaction in 624 runs. Replicated unicast can select express links, but distance-scaled bypass averages 0.965× latency speedup and becomes worse as packets grow. Tree multicast preserves its ordinary XY sharing tree and reports 1.000× with zero express flits. A later directed cost-aware test confirms that the joint path is active: a sparse diagonal-only 2×2 destination completes in 3,500 rather than 4,000 ticks (12.5% faster), whereas a shared three-destination tree retains ordinary XY edges. On the full all-rank broadcast replay the policy also chooses zero express flits, avoiding the measured 19.4% forced-bypass regression.

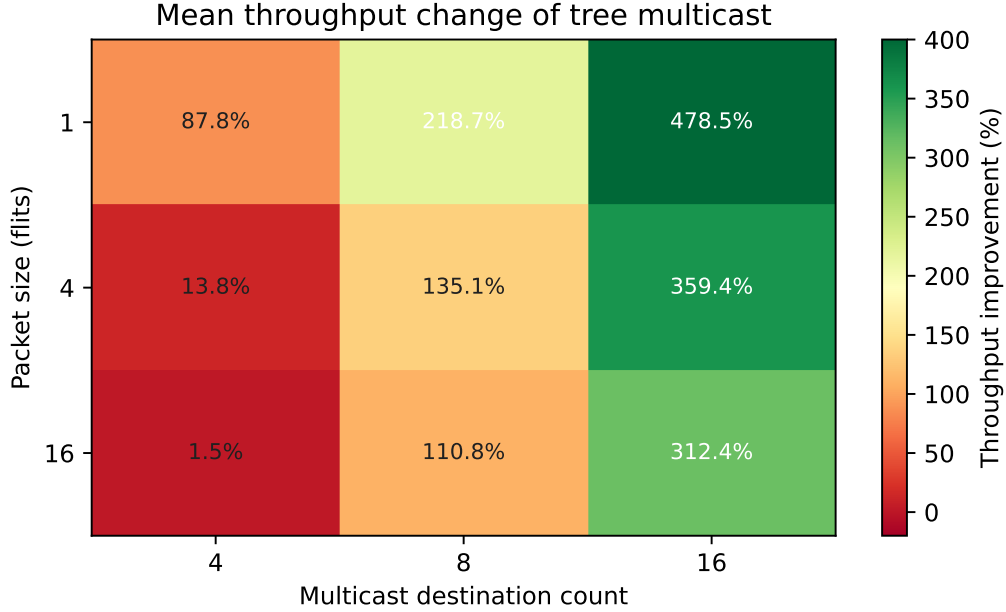


Figure 3: Mean throughput improvement over all offered-load, background, and seed settings for each fanout/packet-size pair.

7 Real-H100 Trace Replay

The checked-in source was captured by actually executing a controlled NCCL collective microbenchmark on eight NVIDIA H100 80GB HBM3 GPUs using CUDA 12.8, PyTorch 2.8.0+cu128, and NCCL 2.27.3. It contains 48 events, including 15 measured broadcasts and 15 measured all-reduces at 1, 4, and 16 MiB. It is not a complete model or optimizer trace. Both byte volume and relative event time are explicitly scaled by 1/1024 for cycle-accurate simulation.

Broadcast events lower to variable-length tree-multicast requests. All-reduce events lower differently: the Router reduction datapath accepts one scalar flit per rank and active lane, so each scaled tensor becomes independent single-flit lanes. Every lane performs a real Router-side reduce followed by tree broadcast; it is not decomposed into ordinary unicast. The lanes are serialized and therefore do not model a vector-width reduction unit.

Table 3: Accepted paired trace-replay results.

Workload	Design	Work units	Source flits	Window ticks	P95 ticks
Broadcast	Mesh XY	30	6,720	17,047,500	643,500
Broadcast	Cost-aware Bypass	30	6,720	17,047,500	643,500
All-reduce lanes	Mesh XY	6,720	53,760	80,638,000	10,000
All-reduce lanes	Mesh Bypass	6,720	53,760	80,638,000	10,000

All 30 broadcast requests complete with 47,040 wire-flit distance. For all-reduce, both designs exactly match 53,760 Router merges, 53,760 rank deliveries, and 147,840 collective Router flits; the network interfaces assert the deterministic eight-rank sum of 36 for every lane. Both complete with 94,080 wire-flit distance. Express usage is zero in Table 3: the all-rank broadcast policy preserves tree sharing, while the all-reduce convergence tree currently contains only ordinary XY parent/child edges. These neutral comparisons must not be presented as bypass acceleration.

Full-group throughput without background traffic

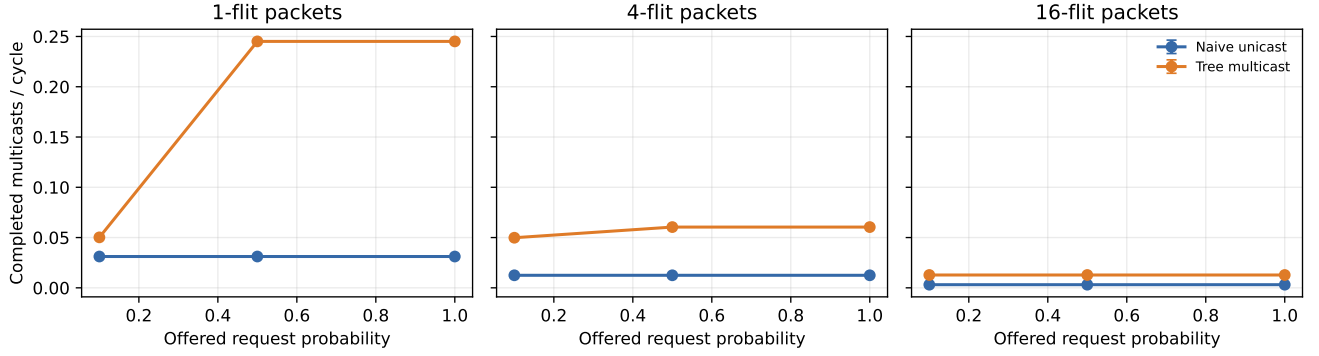


Figure 4: Full-group throughput. Points are means over three seeds; error bars show one standard deviation.

8 Limitations and Threats to Validity

- The performance study currently uses one 4×4 topology and one source router. Corner, edge, and center sources must be separated.
- Performance packet sizes stop at 16 flits; the 64-flit result is currently a correctness result only.
- The broad multicast and bypass sweeps use synthetic traffic. The separate H100 input is a real collective microbenchmark, not a full training trace with compute/communication overlap.
- Offered probabilities 0.5 and 1.0 often collapse to the same measured throughput because outstanding work is bounded. More load points are required to identify saturation precisely.
- The correctness runner uses default VC and buffer settings. A dedicated constrained-resource regression remains necessary.
- The 64-bit destination mask limits a multicast domain to 64 routers.
- Router replication state, extra control logic, and energy/area costs are not yet translated into a physical implementation cost.
- Results describe the current Garnet mechanism and workload contract; they are not measurements of NVIDIA NVLink/NVSwitch hardware.
- Trace all-reduce is serialized scalar-lane execution. It validates reduce/broadcast semantics and counts, not vector reduction bandwidth.

9 Reproducibility

The data snapshot used by this report is stored in `report/data/multicast_paired.csv` and `report/data/multicast_results.json`. Figures and aggregate CSV tables are regenerated with:

```
/path/to/void/bin/python report/scripts/plot_multicast.py
```

The simulator matrices are regenerated from the repository root with:

```
LD_LIBRARY_PATH=/path/to/python/lib \
python3 tests/gem5/lab4/run_multicast_matrix.py \
--jobs 32 --rounds 10 --packet-flits 1 4 16 64
```

```
LD_LIBRARY_PATH=/path/to/python/lib \
python3 tests/gem5/lab4/run_multicast_performance.py --jobs 32
```

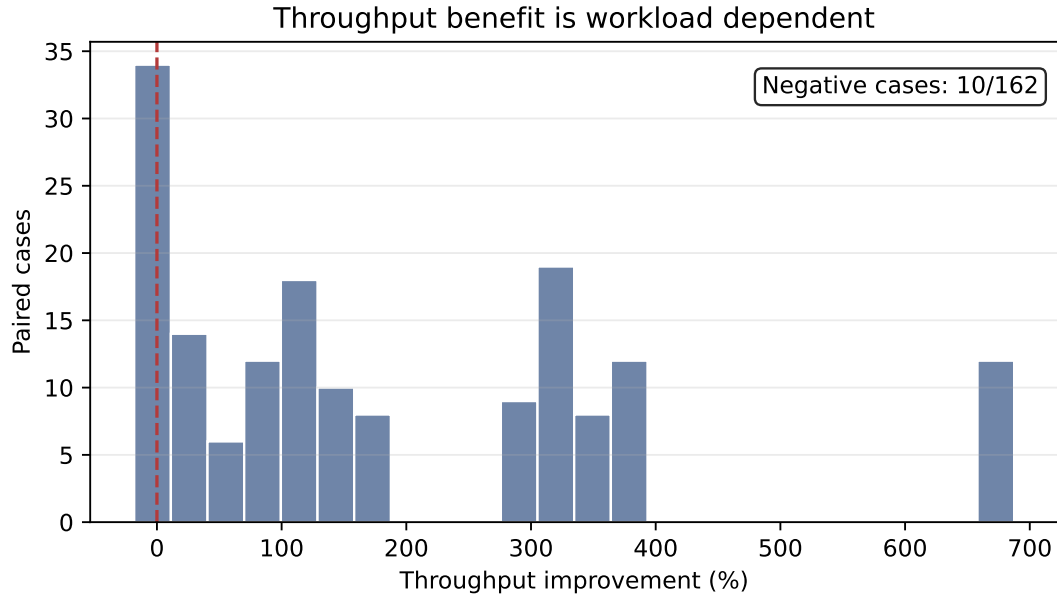


Figure 5: Distribution of throughput improvement. The dashed line marks no change. Negative cases must remain visible in the final conclusion.

The committed trace inputs are independently audited and replayed with:

```
python3 util/lab4_trace/validate_trace.py \
  traces/h100_8gpu_collectives.json
python3 util/lab4_trace/validate_replay.py \
  traces/h100_8gpu_broadcast_replay.json
python3 util/lab4_trace/validate_replay.py \
  traces/h100_8gpu_allreduce_replay.json

python3 tests/gem5/lab4/run_trace_replay.py \
  --gem5 build/Garnet_standalone/gem5.opt \
  --source-trace traces/h100_8gpu_collectives.json \
  --replay traces/h100_8gpu_broadcast_replay.json \
  --output /tmp/lab4-t3 --sim-cycles 30000000

python3 tests/gem5/lab4/run_allreduce_trace_replay.py \
  --gem5 build/Garnet_standalone/gem5.opt \
  --source-trace traces/h100_8gpu_collectives.json \
  --replay traces/h100_8gpu_allreduce_replay.json \
  --output /tmp/lab4-t4 --sim-cycles 100000000
```

The multicast performance snapshot used commit d14545edcb; the integrated trace implementation is commit 2fb5abde72. T3/T4 raw runs explicitly recorded a dirty worktree and base revision b7045e41e0; the report does not relabel them as clean-revision runs. Trace hashes and compact accepted measurements are retained in `report/data/trace_replay_provenance.txt` and `report/data/trace_replay_summary.csv`. The report PDF is built with:

```
cd report
pdflatex multicast_report.tex
pdflatex multicast_report.tex
```

10 Conclusion

The implementation satisfies Topic 3’s multicast and bypass requirements and Topic 4’s trace-based traffic direction. Tree multicast supports arbitrary subsets and multi-flit packets with a runnable replicated-unicast baseline; its gains grow with fanout, but branch synchronization can still reduce throughput. Bypass links are functional and measurably help a sparse directed case, yet distance-scaled and tree-sharing results reject a universal speedup claim. Finally, the committed H100 trace completely drives Garnet broadcast and in-network scalar all-reduce with exact request, flit, merge, delivery, latency, and provenance checks. The appropriate conclusion is therefore a validated interconnection mechanism and replay framework, not completed vector collective hardware or end-to-end AI training acceleration.